

TALLER 3 – INTEGRACIÓN APEX + MONGODB

El siguiente taller tiene una duración máxima de 120 minutos y se realiza en parejas. Puede utilizar la documentación oficial de Oracle APEX, FastAPI y MongoDB, así como sus notas de clase.

El entregable es una aplicación en APEX donde se pueda explorar todo el contenido generado. Suba la solución en la fecha límite indicada por Bloque Neón.

PREPARACIÓN

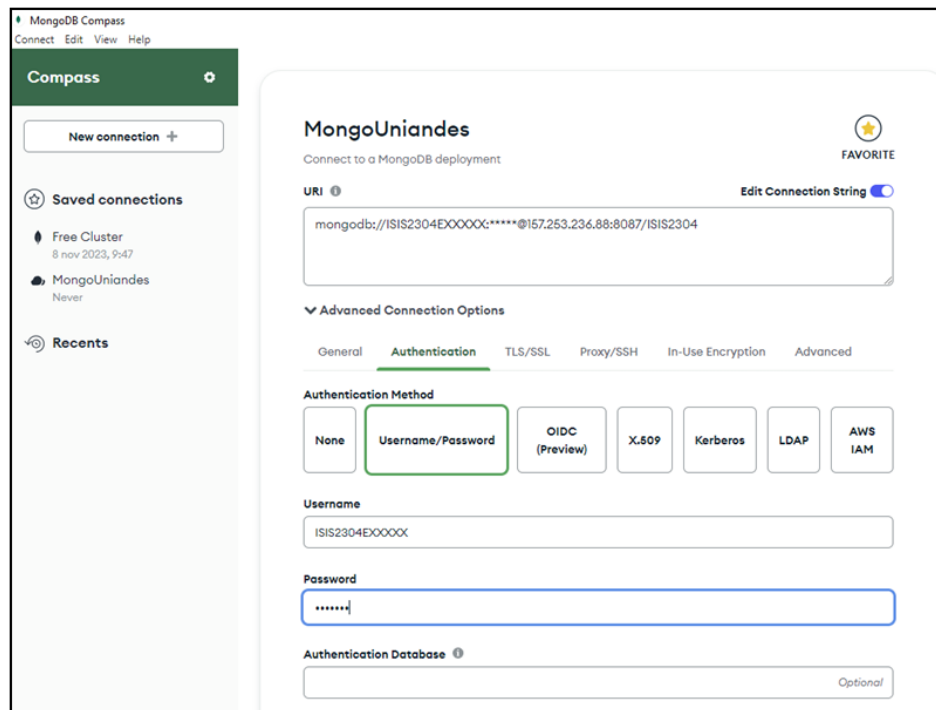
Para el desarrollo de este taller, asegúrese de tener acceso a:

- Su Workspace de Oracle APEX con la base de datos PARRANDEROS cargada y poblada.
- 1. Una cuenta en MongoDB Atlas con el cluster configurado.
- 2. Una cuenta en Render con la API de Python desplegada.
- 3. El repositorio de GitHub con el código base de la API.

La base de datos de Parranderos NoSQL se encuentra en la máquina 157.253.236.88 a la que podrá acceder con sus credenciales de acceso. Puede conectarse desde MongoDB Compass usando la siguiente URI:

mongodb://<usuario>:<contraseña>@157.253.236.88:8087/ISIS2304

Reemplace <usuario> por su nombre de usuario y <contraseña> por su respectiva contraseña para la base de datos proporcionada en el curso. Puede hacerlo a través de la interfaz de conexión de Compass:



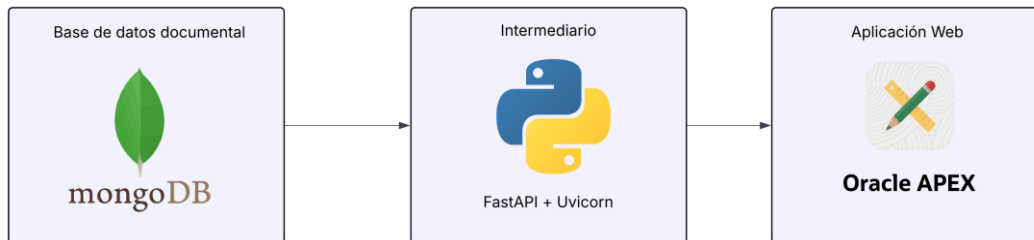
Si su conexión fue exitosa, en su lista de bases de datos en el panel izquierdo debería ver sus bases de datos. Asegúrese de crear las colecciones **comentarios** y **eventos** para solucionar el siguiente taller.

CONTEXTO

La empresa Bebidas de los Andes quiere mejorar su aplicación APEX generando un sistema de comentarios, donde los usuarios puedan dejar comentarios sobre los bares, así como los bares pueden anunciar sus propios eventos. Hasta ahora, toda la información (bares, bebidas y bebedores) se encuentra en Oracle. Sin embargo, el equipo desea que la parte de comentarios y eventos se haga con una capa de datos no estructurada, para permitir comentarios que varían su estructura.

Para esto, el equipo técnico ha decidido integrar MongoDB como base de datos documental. Su equipo debe conectar la aplicación APEX existente con una API REST construida en Python (FastAPI), que actúa como intermediario entre el navegador y MongoDB Atlas.

La arquitectura de la solución



A continuación, se presentan los requerimientos. Responda cada punto con evidencia en su aplicación.

PARTE 1: MODELADO DE DATOS (20 PUNTOS)

Conociendo el contexto de Parranderos, y su necesidad de implementar tanto comentarios sobre el bar, como eventos, diseñe el esquema necesario para soportar estos requerimientos. Incluya un diagrama que indique las decisiones de diseño que tomaron (embeber, referenciar). Una vez diseñado el esquema, responda las siguientes preguntas

4. **(10 puntos)** ¿Qué estrategia se utilizó para modelar cada entidad de Parranderos dentro de la base de datos documental? ¿Cuáles relaciones se modelaron como objetos embebidos y cuáles se modelaron como referencias?
5. **(10 puntos)** ¿Qué estrategia se utilizó para modelar cada entidad dentro de la base de datos documental? ¿Cuáles relaciones se modelaron como objetos embebidos y cuáles se modelaron como referencias?

PARTE 1: COMPLETAR LA API EN PYTHON (25 PUNTOS)

Los endpoints de la API están vacíos. Su tarea es completar las consultas a MongoDB usando PyMongo. Recuerde que PyMongo usa la misma sintaxis de filtros que aprendió en clase:

6. **(5 Puntos)** Completar el endpoint GET /bares/{bar_id}/comentarios para que retorne todos los comentarios del bar cuyo ID llega por la URL. La colección se llama comentarios_bares. El campo de filtro es bar_id.
7. **(5 Puntos)** Completar el endpoint POST /bares/{bar_id}/comentarios para que inserte el documento datos en la colección comentarios_bares. El bar_id y la fecha ya están agregados al documento antes del TODO.
8. **(7 Puntos)** Implementar el endpoint GET /bares/{bar_id}/eventos que retorne todos los eventos del bar cuyo ID llega por la URL. La colección en MongoDB se llama eventos.
9. **(8 Puntos)** Implementar el endpoint POST /bares/{bar_id}/eventos que reciba un documento con la información del evento, le agregue el bar_id y la fecha_creacion, y lo inserte en la colección eventos.

La evidencia de este punto son las capturas de las URLs de la API respondiendo correctamente en el navegador, por ejemplo: <https://su-api.onrender.com/bares/1/comentarios>

PARTE 2: PÁGINA DE DETALLE DEL BAR EN APEX (25 PUNTOS)

Diseñe la arquitectura de navegación para que la aplicación sea intuitiva para el usuario final:

10. **(5 Puntos)** Configurar la navegación desde la página de Cards de bares hacia la página de detalle, pasando el ID del bar seleccionado como parámetro de página (ítem oculto P_BAR_ID).
11. **(5 Puntos)** Mostrar la información del bar seleccionado usando una región con la siguiente consulta sobre Oracle:

```

SELECT NOMBRE, CIUDAD, PRESUPUESTO, CANT_SEDES
FROM   BARES
WHERE  ID = :P_BAR_ID

```

12. **(7 Puntos)** Implementar la carga de comentarios desde la API. En el campo *Execute when Page Loads* de la página, complete el siguiente código JavaScript:

```

const API_BASE = '_____'; // URL de su API en Render
const barId     = parseInt(apex.item('_____').getValue());

function cargarComentarios() {
    fetch(`${API_BASE}/${barId}/comentarios`)
        .then(res => res.json())
        .then(data => {
            const container = document.getElementById('comentarios-
container');
            if (data.length === 0) {
                container.innerHTML = '<p>No hay comentarios aún. ¡Sé el
primero!</p>';
                return;
            }
            container.innerHTML = data.map(c => `
                <div style="border:1px solid #ddd; border-radius:8px;
padding:12px; margin-bottom:10px;">
                    <strong>${c.autor || 'Anónimo'}</strong>
                    <span style="color:#888; font-size:0.85em; margin-
left:10px;">
                        ${c.date ? new Date(c.date).toLocaleDateString('es-
CO') : ''}
                    </span>
                    <p style="margin-top:6px;">${c.texto}</p>
                </div>
            `).join('');
        })
        .catch(err => {
            document.getElementById('comentarios-container').innerHTML =
                '<p style="color:red;">Error cargando comentarios.</p>';
        });
}

```

13. **(8 Puntos)** Implementar el envío de comentarios. Agregue la función `enviarComentario()` y conéctela a un botón mediante un Dynamic Action. La función debe hacer un POST a la API con los campos `bar_id`, `texto` y `autor`.

```
function enviarComentario() {
  const texto = apex.item('_____').getValue();
  if (!texto.trim()) {
    apex.message.showErrors([{ message: 'El comentario no puede estar vacío.' }]);
    return;
  }

  fetch(`${API_BASE}/${barId}/comentarios`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      bar_id: parseInt(barId),
      texto: texto,
      autor: apex.env.APP_USER // usuario de sesión APEX
    })
  })
  .then(res => res.json())
  .then(() => {
    apex.item('_____').setValue("");
    cargarComentarios(); // refresca la lista
  })
  .catch(() => {
    apex.message.showErrors([{ message: 'Error enviando el comentario.' }]);
  });
}
```

Evidencia: Capturas de la página mostrando el detalle del bar, los comentarios cargados desde MongoDB y un comentario recién enviado.

Parte 3: Eventos de Bares (30 Puntos)

Implemente una página en APEX que muestre los eventos de un bar y permita registrar nuevos. A diferencia de los comentarios (que tienen siempre la misma estructura), cada evento puede tener campos completamente distintos según su tipo.

Ejemplo de documentos en la colección eventos:

```
// Concierto
{
  "bar_id": 2,
  "nombre": "Noche de Jazz",
```

```
"artista": "Trío Bogotá",
"cover": 15000,
"cupos": 80
}

// Happy hour - campos completamente distintos
{
  "bar_id": 2,
  "nombre": "Happy Hour Martes",
  "descuento": "2x1 en cervezas",
  "hora_inicio": "17:00",
  "hora_fin": "19:00"
}
```

14. **(15 Puntos)** Agregue una sección de Eventos en la página de detalle del bar que cargue y muestre los eventos desde el endpoint `GET /bares/{bar_id}/eventos` de su API.
1. **(15 Puntos)** Permita registrar un nuevo evento. El formulario debe tener al menos nombre (obligatorio) y dos campos opcionales que varían según el tipo de evento (por ejemplo: artista y cover para conciertos, o descuento y hora_inicio para happy hours). El documento enviado al POST debe incluir únicamente los campos que el usuario completó.

Evidencia: Capturas mostrando dos eventos con estructuras distintas almacenados en MongoDB y visualizados en APEX.

Entregables

- Entregue en BN un .zip o .sql de la exportación de su aplicación de APEX.
- Enlace al repositorio GitHub con el archivo mongo-api.py completo.
- URL pública de la API desplegada en Render.
- Informe